

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Shaik. Sameena(192372264)

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

Code of Conduct:

I Shaik. Sameena(192372264) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: Servlet-Based Student Registration System

A **Servlet** is a Java program that runs on a web server and handles requests from clients, processes data, and generates responses. A servlet-based student registration system can be used to receive student details from an HTML form and process them on the server.

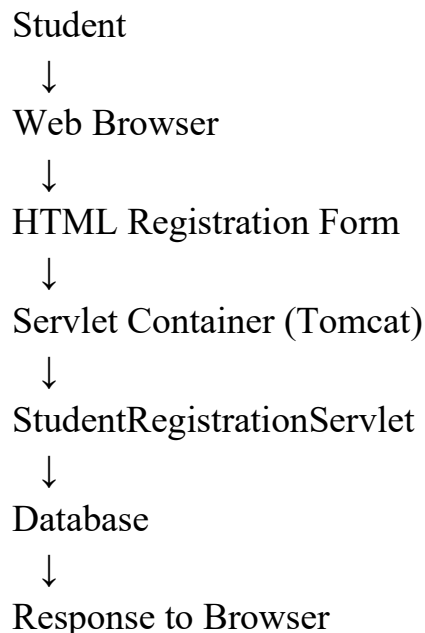
1. Architecture of a Servlet-Based Application

The basic architecture consists of:

Client → Web Browser → Web Server/Servlet Container → Servlet → Database → Response

- **Client:** Student enters registration details using an HTML form.
- **Web Browser:** Sends the form data to the server.
- **Servlet Container:** For example, Apache Tomcat receives the request and manages the servlet.
- **Servlet:** Processes the request using Java code.
- **Database:** Stores student registration information.
- **Response:** The servlet sends an HTML response back to the browser.

Simple Architecture



2. HTML Registration Form

`<form action="StudentRegistrationServlet" method="post">`

Name:

`<input type="text" name="name"><br><br>`

Register Number:

```
<input type="text" name="regno"><br><br>
```

Email:

```
<input type="email" name="email"><br><br>
```

Course:

```
<input type="text" name="course"><br><br>
```

```
<input type="submit" value="Register">
```

```
</form>
```

The form sends the student's details to the servlet using the **POST** method.

3. Servlet Implementation Using GET

```
import java.io.*;
```

```
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;
```

```
public class StudentRegistrationServlet extends HttpServlet {
```

```
    protected void doGet(HttpServletRequest request,  
                           HttpServletResponse response)  
        throws ServletException, IOException {
```

```
        response.setContentType("text/html");
```

```
        PrintWriter out = response.getWriter();
```

```
        String name = request.getParameter("name");  
        String regno = request.getParameter("regno");  
        String email = request.getParameter("email");  
        String course = request.getParameter("course");
```

```
        out.println("<h2>Student Details</h2>");  
        out.println("Name: " + name + "<br>");  
        out.println("Register Number: " + regno + "<br>");  
        out.println("Email: " + email + "<br>");  
        out.println("Course: " + course);
```

```
}  
}
```

The doGet() method receives data through the URL query string.

Example:

StudentRegistrationServlet?name=Ravi®no=101&email=ravi@gmail.com

4. Servlet Implementation Using POST

```
import java.io.*;
```

```
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;
```

```
public class StudentRegistrationServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest request,  
                           HttpServletResponse response)  
        throws ServletException, IOException {
```

```
        response.setContentType("text/html");
```

```
        PrintWriter out = response.getWriter();
```

```
        String name = request.getParameter("name");  
        String regno = request.getParameter("regno");  
        String email = request.getParameter("email");  
        String course = request.getParameter("course");
```

```
        out.println("<h2>Registration Successful</h2>");  
        out.println("Name: " + name + "<br>");  
        out.println("Register Number: " + regno + "<br>");  
        out.println("Email: " + email + "<br>");  
        out.println("Course: " + course);
```

```
    }  
}
```

The doPost() method receives the form data in the HTTP request body rather than displaying it in the URL.

5. Servlet Life Cycle

The servlet life cycle consists mainly of three methods:

init()

The `init()` method is called **only once** when the servlet is created.

```
public void init() {  
    System.out.println("Servlet initialized");  
}
```

It can be used for initialization tasks such as creating resources or establishing required connections.

service()

The `service()` method is called whenever a request is received.

Client Request

↓
`service()`
↓
`doGet()` / `doPost()`

For an HTTP servlet, the `service()` method determines whether the request should be handled by `doGet()`, `doPost()`, or another HTTP method.

destroy()

The `destroy()` method is called before the servlet is removed from service.

```
public void destroy() {  
    System.out.println("Servlet destroyed");  
}
```

It is used for cleanup operations.

6. GET vs POST

Feature	GET	POST
Data location	URL	Request body
Visibility	Visible in URL	Not normally visible in URL
Suitable for	Retrieving data	Submitting data
Data size	More limited	Suitable for larger data
Sensitive information	Not recommended	More appropriate

Suitable Method for Sensitive Data

POST is more appropriate for handling sensitive form data because the data is sent in the request body instead of being placed in the URL. However, POST alone does not encrypt data. For truly sensitive information, the application should also use **HTTPS**, server-side validation, authentication, and secure session management.

Question 2: Session Management Using HttpSession and Cookies

HTTP is a **stateless protocol**, meaning that each request from a browser is treated independently. A web application therefore needs session tracking to remember that a user has logged in while moving between different pages.

1. Session Management Using HttpSession

HttpSession allows the server to store information about a particular user across multiple requests.

Login Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if (username.equals("admin") &&
            password.equals("1234")) {

            HttpSession session = request.getSession();

            session.setAttribute("username", username);

            response.sendRedirect("home");
        }
        else {
            response.getWriter().println("Invalid Login");
        }
    }
}
```

After successful authentication, the username is stored in the session.

2. Accessing Session Data

Another servlet can retrieve the stored information.

```
HttpSession session = request.getSession(false);
```

```
if (session != null) {
```

```
    String username =
```

```
        (String) session.getAttribute("username");
```

```
    response.getWriter().println(
```

```
        "Welcome " + username);
```

```
}
```

```
else {
```

```
    response.getWriter().println("Please login first");
```

```
}
```

The same session can be used across multiple pages.

3. Logout

The session can be terminated using:

```
HttpSession session = request.getSession(false);
```

```
if (session != null) {
```

```
    session.invalidate();
```

```
}
```

```
response.sendRedirect("login.html");
```

invalidate() removes the session and logs the user out.

4. Session Tracking Using Cookies

A **cookie** is a small piece of information stored by the browser. A servlet can create a cookie using the Cookie class.

```
Cookie =
```

```
    new Cookie("username", "admin");
```

```
cookie.setMaxAge(60 * 60);
```

```
response.addCookie(cookie);
```

The browser stores the cookie and sends it back with subsequent requests.

The servlet can read the cookie using:

```

Cookie[] cookies = request.getCookies();

if (cookies != null) {

    for (Cookie : cookies) {

        if (cookie.getName().equals("username")) {

            String username =
                cookie.getValue();

            System.out.println(username);
        }
    }
}

```

5. How Session Tracking Works

The process is:

User Login



Server Authenticates User



HttpSession Created



Session ID Generated



Session ID Sent to Browser



Browser Sends Session ID
with Subsequent Requests



Server Identifies User Session

Servlet containers commonly use a cookie such as JSESSIONID to associate the browser with the server-side session.

6. Session vs Cookies

Feature	HttpSession	Cookies
Storage	Mainly server-side	Client/browser-side

Feature	HttpSession	Cookies
Data exposure	Less directly exposed	Stored on client
Data capacity	Better for application session data	Limited
Security	Generally preferable for session state	Requires secure cookie settings
Lifetime	Can be controlled by session timeout	Can be session or persistent
Common use	Login/session information	Preferences, identifiers, tracking

Conclusion

For login authentication, **HttpSession is generally preferable** because important session information can remain on the server. A secure session cookie can be used to carry only the session identifier. Cookies can additionally be used for non-sensitive preferences or other client-side information.

For security, applications should use HTTPS and appropriate cookie attributes such as Secure, HttpOnly, and SameSite.

\

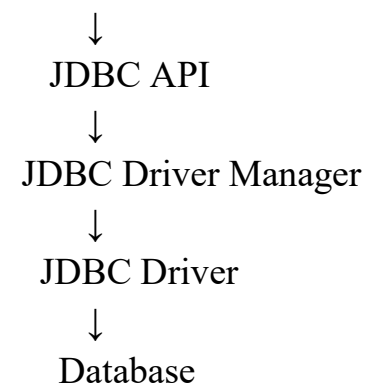
Question 3: JDBC-Based Product Management Application

JDBC (Java Database Connectivity) is a Java API that allows Java applications to connect to relational databases, execute SQL statements, and retrieve results. An e-commerce application can use JDBC to store and retrieve product information such as product ID, product name, price, and quantity.

1. JDBC Architecture

The basic JDBC architecture is:

Java Web Application



Main Components

1. **JDBC API** – Provides classes and interfaces for database operations.
2. **DriverManager** – Manages database drivers and establishes connections.
3. **JDBC Driver** – Communicates between Java and the database.
4. **Connection** – Represents a connection to the database.
5. **Statement / PreparedStatement** – Executes SQL commands.
6. **ResultSet** – Stores the results returned by a SELECT query.

2. Product Table

Example SQL table:

```
CREATE TABLE products (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    price DECIMAL(10,2),  
    quantity INT  
);
```

3. JDBC Steps

The main steps involved in JDBC connectivity are:

Step 1: Import JDBC Classes

```
import java.sql.*;
```

Step 2: Load/Register the Driver

Modern JDBC drivers are generally discovered automatically when available on the classpath. Explicit driver loading may also be used with some setups.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Step 3: Establish Connection

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/shop",  
    "root",  
    "password"  
);
```

Step 4: Create PreparedStatement

PreparedStatement is preferred for user-supplied values because it helps prevent SQL injection.

```
String sql =  
    "INSERT INTO products VALUES (?, ?, ?, ?)";
```

```
PreparedStatement ps =  
    con.prepareStatement(sql);
```

Step 5: Insert Product Data

```
ps.setInt(1, 101);  
ps.setString(2, "Laptop");  
ps.setDouble(3, 55000.00);  
ps.setInt(4, 10);
```

```
ps.executeUpdate();
```

executeUpdate() is used for INSERT, UPDATE, and DELETE operations.

4. Retrieving Product Data

```
String sql = "SELECT * FROM products";
```

```
PreparedStatement ps =  
    con.prepareStatement(sql);
```

```
ResultSet rs = ps.executeQuery();
```

```
while (rs.next()) {
```

```
    int id = rs.getInt("id");  
    String name = rs.getString("name");
```

```
double price = rs.getDouble("price");
int quantity = rs.getInt("quantity");
```

```
System.out.println(
    id + " " + name + " " +
    price + " " + quantity);
}
```

executeQuery() is used for SELECT operations, and the ResultSet contains the returned rows.

5. Complete JDBC Example with Exception Handling

```
import java.sql.*;
```

```
public class ProductDAO {
```

```
    public static void main(String[] args) {
```

```
        String url =
            "jdbc:mysql://localhost:3306/shop";
```

```
        String username = "root";
        String password = "password";
```

```
        String insertSQL =
            "INSERT INTO products " +
            "(id, name, price, quantity) " +
            "VALUES (?, ?, ?, ?)";
```

```
        String selectSQL =
            "SELECT * FROM products";
```

```
        try (Connection con =
            DriverManager.getConnection(
                url, username, password);
```

```
            PreparedStatement insert =
                con.prepareStatement(insertSQL)) {
```

```

insert.setInt(1, 101);
insert.setString(2, "Laptop");
insert.setDouble(3, 55000);
insert.setInt(4, 10);

insert.executeUpdate();

try (PreparedStatement select =
    con.prepareStatement(selectSQL);
    ResultSet rs =
        select.executeQuery()) {

    while (rs.next()) {

        System.out.println(
            rs.getInt("id") + " " +
            rs.getString("name") + " " +
            rs.getDouble("price") + " " +
            rs.getInt("quantity"));
    }
}

} catch (SQLException e) {

    System.out.println(
        "Database error: " + e.getMessage());
}
}
}

```

6. Exception Handling

JDBC operations can generate SQLException. Therefore, database operations should be placed inside a try-catch block.

```

try {
    // Database operations
}
catch (SQLException e) {
    System.out.println(e.getMessage());
}

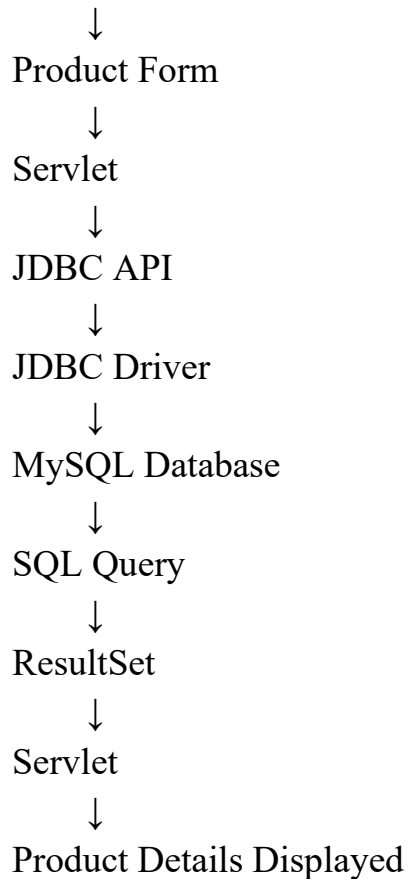
```

```
}
```

Using **try-with-resources** is better because connections, statements, and result sets are automatically closed after use.

7. Overall Working

E-Commerce Web Page



Conclusion

JDBC provides a standard method for connecting Java web applications to databases. The application can use Connection, PreparedStatement, and ResultSet to insert and retrieve product information. Using PreparedStatement, exception handling, and try-with-resources makes the application safer, more reliable, and easier to maintain.